

ABSTRACT

TISIOPS: AI DEVOPS CLOUD ENGINEER

Submitted By

Name	University Roll Number	Email ID
Parampreet Singh	2310991088	parampreet1088.be23@chitkara.edu.in

Faculty Coordinator

Prof. (Dr.) Rajat Takkar

Institute

Chitkara Institute of Engineering and Technology (CUIET)

Chitkara University, Punjab

ABSTRACT

TisiOps: AI DevOps Cloud Engineer is a proposed web-based platform for helping developers deploy, manage, monitor, and recover cloud-hosted applications through a controlled natural-language interface. The idea comes from a pretty common problem: an application may work on a developer's laptop, then fail during deployment because of a missing environment variable, an occupied port, a Docker build error, a wrong server permission, or a service that simply did not start. For a small team, finding the cause means jumping between cloud dashboards, SSH terminals, build logs, containers, and random documentation. It gets tiring fast.

The problem is important because deployment failures directly affect availability, customer trust, and development time. A developer can lose hours reading hundreds of log lines just to notice one database URL or port mismatch. In many existing DevOps tools, logs and monitoring information are available, but the user still has to interpret everything and decide which command is safe to run. Generic AI assistants can suggest commands, but they may not see the actual server state, recent deployment actions, or the live health of a container. That gap is where mistakes happen, honestly.

TisiOps addresses this limitation by treating failure recovery as an evidence-based workflow rather than a one-shot chatbot answer. It gathers deployment logs, application logs, container status, port availability, CPU and memory signals, endpoint health checks, and the recent action history for a selected project. These signals are combined into a clear incident view. The system then explains the likely cause in simple language and prepares a recovery plan. Anyway, the plan is not executed automatically when it can change live infrastructure; the user must review and approve it first.

The proposed method has four connected stages. First, TisiOps receives a deployment request or a failure event and records the current deployment state. Second, it performs structured checks on logs, runtime configuration, active containers, mapped ports, and server resources. Third, an AI-assisted diagnosis component compares these verified signals with known operational patterns and produces ranked recovery options. Finally, after user approval, the platform can carry out a limited action such as restarting the identified service, updating a port mapping, redeploying a failed build, or showing the exact command needed for manual repair. Each step is logged, so the user can see what changed and why.

Artificial intelligence is used in TisiOps as a guided reasoning layer, not as an unrestricted system administrator. The AI classifies the user's request, groups related log messages, connects them with health-check results, and rewrites technical errors into a short explanation that a developer can act on. It can also draft a recovery plan with a reason, expected outcome, and risk for every proposed step. This is useful when an error is spread across several places, such as a failed Docker build followed by an unhealthy container and a blocked endpoint. The final execution decision still belongs to the user.

For example, if a Node.js service fails because port 3000 is already occupied, TisiOps should not just say “restart the server.” It checks which process or container owns the port, relates that result to the failed deployment log, and offers specific options: stop the conflicting service, change the deployment port, or cancel the action. To be fair, the platform cannot guarantee that every error has one perfect fix. Its contribution is making the recommendation traceable to current evidence instead of generating a plausible but unverified answer.

The novelty of TisiOps lies in its evidence-grounded and approval-gated DevOps assistance. Existing dashboards mainly display raw signals, while many AI tools produce advice without a reliable connection to the target infrastructure. TisiOps links a proposed command or recovery action to the exact logs, health checks, and deployment states that support it. It also separates diagnosis from execution: the system can reason about the issue, but destructive or production-affecting operations remain visible and require approval. This makes the platform useful for developers who understand their code but may not be comfortable with cloud operations.

The expected result is a reduction in the time needed to identify common deployment failures, fewer unsafe trial-and-error commands, and a more understandable cloud-management process for student projects, startups, and small development teams. The platform can also retain incident histories, which may help users recognise repeated failures and avoid them in later deployments (a small detail, but quite useful in practice). Future work may extend the system to multi-cloud environments, automated rollback suggestions, and preventive checks before a deployment is released.

Keywords: Artificial Intelligence, AI-Assisted DevOps, AI-Based Incident Diagnosis, Natural-Language Infrastructure Management, Cloud Engineering, Deployment Recovery, Log Analysis, Docker, Server Monitoring, Safe Infrastructure Management, Approval-Gated Execution